
Boundary-Cadence Rollback for Code Generation: A Pilot Comparison Against Monitor-Guided Decoding on Rust

Leo Wang

Department of Computer Science
University of Chicago
psixyzt@uchicago.edu

Abstract

Language-model code generation is increasingly paired with static analysis to constrain the output space. Two complementary approaches have emerged: pre-emptive masking at the token level (Monitor-Guided Decoding; 1) and post-hoc rollback at coarser granularity (e.g. self-refine 5, reflection 10). The two have not been measured side-by-side in the same language under the same evaluation metric. We report a pilot comparison on MultiPL-E HumanEval Rust [4] across four arms: (i) raw generation, (ii) Monitor-Guided Decoding ported to rust-analyzer, (iii) rollback at statement-boundary cadence using `cargo check`, and (iv) MGD combined with rollback. Three findings emerge: rollback’s benefit is strongly model-dependent — positive for mid-strength models, neutral for weak ones, *negative* for the strongest baseline. Monitor-Guided Decoding, which improves Java code generation in published work [1], *regresses* pass@1 by 30–47 percentage points on Rust at 1.5B parameters. Combining MGD with rollback does not recover. A diagnostic isolates one implementation-side cause (`multilspy` returns LSP labels rather than identifiers as completion text) but fixing it does not reverse the regression — with cleaner completion sets the MGD mask becomes *more* restrictive and the small model’s failure surface widens. We argue this is consistent with the MGD paper’s success conditions being tied to Java’s more permissive LSP and to the dereference-only task framing the paper evaluates on, neither of which transfers unmodified to whole-function Rust generation.

Status note. This is a pilot. Several experiments described below are incomplete or rely on capped/subsampled data. Sections marked

[INCOMPLETE] ...

flag what is missing and why; the discussion section qualifies the strength of each claim accordingly.

1 Introduction

LLM code generation has reached the point where naive autoregressive sampling routinely produces compiling, sometimes correct, code for short programming tasks. The frontier has shifted toward (a) making generation reliably correct on harder tasks, and (b) doing so without retraining the underlying model. A productive line of work has paired LLMs with the static-analysis tools already shipped with modern programming languages — type checkers, language servers, compilers — consumed during decoding to either constrain what the model can emit or to detect and react to its mistakes.

Two complementary approaches have crystallized. *Pre-emptive* methods mask the LM’s logits at every token so that only outputs consistent with the static analyzer’s predictions are sampled. Monitor-Guided Decoding (MGD; 1) is the canonical instance: at every dereference (a token following `.`), MGD queries an LSP for the legal completions of the expression’s type and zeroes out logits for tokens that cannot extend any of those completions. *Reactive* methods let the LM generate freely until a checker detects an error, then roll back to a known-good state. Compiler-in-the-loop systems [8, 6] and natural-language self-refinement [5, 10] are both reactive in this sense.

The two designs probe different points in a precision-recall tradeoff. Pre-emptive masking is dense (it intervenes at every trigger) and prevents bad tokens from being committed at all. Reactive rollback is sparse (it intervenes only when a checker fires) and lets the LM commit to longer paths, only undoing them when a clear error is detected. To our knowledge no prior work has measured the two side-by-side on the same benchmark in the same language.

This pilot does that comparison on Rust, using `rust-analyzer` for the LSP, `cargo check` for the compile-time checker, and MultiPL-E HumanEval Rust as the test set. We implement all four arms (raw, MGD, rollback, MGD+rollback) and report pass@1 and compilation rate. We also implement two rollback variants — *mute* (truncate the buffer and re-prompt) and *instruct* (truncate, then re-prompt with the cargo error appended as a comment) — to separate the value of undoing a bad path from the value of telling the model what went wrong.

Our results disagree sharply with the MGD paper’s reported Java results, in a direction we did not expect. We argue this is a finding about the *conditions under which pre-emptive masking helps*, rather than a refutation of MGD itself: the same algorithm that delivers +18–25% relative compilation-rate gains on Java DOTPROMPTS reduces Rust pass@1 by 30–47 pp on our pilot. We diagnose one real port-side bug (`multispy`’s `completionText` returns LSP labels rather than bare identifiers; §5), fix it, and observe that the gap does not close. We discuss four orthogonal axes that differ between our setup and the original MGD evaluation (language, LSP, task framing, sampling regime).

Contributions.

1. A reproducible implementation of all four (raw, MGD, rollback, MGD+rollback) arms for Rust, built on `cargo check` (for rollback verification) and a port of MGD’s dereferences monitor to `rust-analyzer` via `multispy`.
2. A three-model, 30-problem pilot showing rollback’s effect is baseline-quality-dependent: +6.7 pp compilation rate on weak-to-mid models, −6.7 pp on a strong baseline.
3. A five-arm, $n=15$ pilot on Qwen2.5-Coder-1.5B-Instruct showing MGD regresses pass@1 by 30–47 pp on Rust, and that combining MGD with rollback does not recover.
4. A diagnosis of one MGD-port bug (`multispy`’s `completionText` returns labels with `()` and trait-disambiguation suffixes attached, not bare identifiers); a patch; and the observation that the patch does *not* reverse the regression on this small-model Rust pilot.
5. A characterization of three orthogonal factors that plausibly explain why MGD’s Java result does not transfer.

2 Background

2.1 Monitor-Guided Decoding (MGD)

Agrawal et al. [1] introduce a framework that pairs an LLM with a programming-language language server during decoding. A *monitor* is a stateful interface that observes generated tokens and, at predefined *trigger points*, queries a static analyzer for permissible completions, converting the result to a binary mask over the LM’s vocabulary. Formally, the guided distribution is:

$$p(x_{n+1} \mid x_{1:n}) = \begin{cases} \text{softmax}(\ell) & \text{wait state} \\ \text{softmax}(\ell \oplus m) & \text{active state, mask } m \end{cases}$$

where ℓ is the raw logit vector and $\oplus$ sets masked positions to $-\infty$. The canonical instantiation is the *dereferences monitor*: on emitting a `.`, query the LSP for valid completions of the type of the preceding expression, then mask the vocabulary so that only tokens prefixing those completions can be sampled.

Evaluation. MGD was evaluated on DOTPROMPTS, a benchmark of 10,538 Java dereference-completion prompts derived from PRAGMATICCODE (100 real-world Java repositories). Each prompt is a Java method truncated at a `.` operator; the task is to generate the next ~ 30 tokens. Metrics include compilation rate (CR; drop the regenerated method back into the original repo and run `javac`) and next-identifier match (NIM). MGD reported gains of +18–25% relative CR across all five evaluated model scales (CodeGen-350M, CG-2B, CG-6B, SantaCoder-1.1B, text-davinci-003).

Reported limitations. The MGD paper itself notes (i) Java-focused evaluation, (ii) no backtracking when the LSP returns an empty completion set (the run is abandoned), (iii) trigger-point intervention only (errors between triggers are not caught), and (iv) compilability $\neq$ functional correctness.

2.2 Reactive and Self-Refine approaches

Madaan et al. [5] and Shinn et al. [10] take the opposite design choice: let the LM generate freely, then iterate by feeding back natural-language critiques. Olausson et al. [6] apply compile-error-as-critique to code generation. AlphaCodium [8] extends this to test-time iteration with hand-designed agentic flow. All of these are post-hoc: they verify entire generations, then condition a fresh generation on the verifier’s output.

2.3 Other constrained-decoding approaches

A separate line of work [9, 7, 12, 11, 3] constrains the LM at the token or grammar level, but with *syntactic* rather than semantic constraints. PICARD [9] parses partial outputs against the target SQL grammar; SynCode [11] generalizes to arbitrary context-free grammars. These cannot enforce type correctness or identifier resolution against a host program; MGD’s contribution is making those semantic constraints accessible at decoding time.

2.4 LSP-based static analysis and multilspy

The MGD authors released `multilspy` [2], a Python wrapper around the Language Server Protocol that supports Java (Eclipse JDT.LS), Python (Jedi), C# (OmniSharp), and Rust (`rust-analyzer`). It handles lifecycle, document state, and completion queries. `multilspy` is the bridge between the LLM and the LSP that makes MGD-style monitors implementable across languages.

3 Method

3.1 The four arms

Table 1: The four (five, counting two rollback sub-variants) arms in this comparison.

	(i) raw	(ii) MGD	(iii) rollback	(iv) MGD+rollback
Signal	none	LSP completions	<code>cargo check</code>	both
Granularity	file	sub-token, at <code>.</code>	block (depth-0 <code>;</code> , <code>}</code>)	both
Action	none	logit mask	truncate + re-prompt	both
LSP/checker cost	none	high	low	high

Arms (iii) is split into two variants:

- **(iii-mute)** On a blocking diagnostic, truncate to the last clean checkpoint and re-prompt with (original prompt) + (verified prefix).
- **(iii-instruct)** Same truncation, but re-prompt is (original prompt) + (verified prefix) + `format_error(diag)` where `format_error` renders the cargo diagnostic as a comment block. Closer to Self-Refine in the loop structure.

3.2 Rollback algorithm

The runner is producer–consumer (Algorithm 1). A single producer pulls tokens from the LM; a single consumer drains a FIFO queue of in-flight LSP-tier checks. `Code.append` is sync (it just

grows the buffer); `Code.check` is async (it runs `cargo check` on the snapshot and updates the state machine `INACTIVE/OK/ERROR`). A check is scheduled only when the just-appended token completes a statement at brace depth 0 of the function body. On a blocking diagnostic, pending checks are cancelled, the buffer truncates to the last OK-marked checkpoint, and the LM is re-prompted.

Algorithm 1 Producer–consumer rollback (simplified)

```
tasks <- empty FIFO
code <- Code(prompt)
llm.set_prompt(prompt)

while true:
  if llm.has_next():
    token <- llm.next()
    code.append(token)
    if code.at_depth0_boundary():
      tasks.append(spawn code.check())
  while tasks and tasks.head().done():
    result <- tasks.pop().result()
    if result.is_error:
      cancel pending tasks; tasks.clear()
      llm.abort_current_stream()
      llm.set_prompt(prompt
        + code.up_to(code.last_ckpt)
        + format_error(result.msg))
      code.rollback()
      break
  if not llm.has_next():
    if tasks.empty(): return code.content
    await tasks.head()
```

3.3 Verification signal at boundaries

For each boundary check the runner wraps the partial buffer with an `unreachable!()` shim so the function type-checks despite the body being incomplete:

```
{prompt} // function signature ending with '{'
{partial_body}
  unreachable!() // never type, coerces to any return
}
fn main() {}
```

This lets us run `cargo check` on partial code. It also has the side effect of *hiding* return-type errors (the `!` type satisfies any return type), which we discuss as a limitation in §7.

Error classification. A cargo diagnostic is one of three categories: `INCOMPLETE` (syntax-error, E0282 type annotations needed, messages containing “expected” / “unterminated” / “unclosed” / “unexpected eof”); `NONBLOCKING` (warnings); or `BLOCKING` (everything else). Only `BLOCKING` triggers rollback.

3.4 Port of MGD’s dereferences monitor

Algorithmically a direct port of `microsoft/monitors4codegen’s DereferencesMonitor`. Three implementation differences:

1. **Trigger:** the reference monitor fires when `text[-1] == "."`. Modern tokenizers (Qwen’s BPE) encode `.iter` as a single token, so a one-character trigger almost never fires. We relax to “text ends with `<identifier-chars>`” (zero or more identifier chars after the dot) and filter the legal set by the already-emitted prefix on the next call. We reject `..` (range) and `<digit>.<digit>` (float literal) explicitly.
2. **Closer:** the same `unreachable!()` shim from §3.3 is appended after the cursor so rust-analyzer sees a parseable function.

3. **Vocab trie:** the reference uses a pygtrie over vocab tokens; we use a precomputed `token_id → decoded_string` table and linear scan. Asymptotically worse, negligible at pilot scale.

3.5 Implementation

The Ollama-based pilot (§4.1) uses HTTP streaming with abort. The HF-transformers pilot (§4.2) uses `model.generate` with a `LogitsProcessor` subclass that applies the mask per token. We initially attempted vLLM but found that vLLM v1’s engine-subprocess architecture serializes the `LogitsProcessor`’s closure-captured state across IPC, breaking the monitor: state mutations in the engine never reach the main process, and the async-driven LSP client object cannot be safely pickled. We document this for reproducibility in appendix A.

4 Experiments

4.1 Rollback only (Ollama, 3 models)

Setup. Three models from the open-source code-LM sweet spot (weak, mid, mid-strong), served by local Ollama; 30 problems from MultiPL-E HumanEval Rust; nucleus sampling ($T=0.2$, $top_p=0.95$); 60 s wall-budget per problem; `max_tokens=512`.

Result. Table 2.

Table 2: Rollback-only pilot, $n=30$ per cell. Bold marks the largest deltas (positive or negative) vs raw within each model.

Model	arm	pass@1	compile	mean wall (s)	rollbacks	Wilcoxon p (wall)
nemotron-3-nano:4b	raw	0.533	0.600	0.46	0	—
	mute	0.533	0.667	0.95	69	< 0.0001
	instruct	0.500	0.600	1.04	21	0.0001
mistral-small3.2:24b	raw	0.667	0.900	0.75	0	—
	mute	0.700	0.967	0.94	16	0.25
	instruct	0.633	0.967	0.89	1	0.43
qwen3.5:9b	raw	0.833	0.967	0.88	0	—
	mute	0.733	0.900	1.41	17	0.02
	instruct	0.833	0.933	1.18	7	0.0003

Headline. Rollback’s effect depends on baseline strength:

- **Weak baseline (nemotron, raw = 53.3% / 60.0%).** Rollback recovers compile rate by 6.7 pp; pass@1 flat. Wall-clock penalty is significant ($p < 10^{-4}$) because the model frequently oscillates (37% of problems hit the 8-rollback cap on mute).
- **Mid baseline (mistral, raw = 66.7% / 90.0%).** Best case for rollback: +3.3 pp pass, +6.7 pp compile, non-significant wall penalty.
- **Strong baseline (qwen, raw = 83.3% / 96.7%).** Rollback *hurts*: −10.0 pp pass on mute, −6.7 pp compile. The model’s false-positive rate ($\sim 5\%$ on this evaluation) exceeds the rate of bugs the model would naturally make.

This is the first paper-shaped finding: rollback verification is a baseline-quality-dependent tradeoff, not a Pareto improvement. §6.1 returns to why.

4.2 Five-arm with MGD (HF transformers, Qwen2.5-Coder-1.5B-Instruct)

Setup. HF transformers with custom `LogitsProcessor` for the MGD arms; rust-analyzer 0.3.x via `multilspy`; same 30-problem subset of MultiPL-E HumanEval Rust plus `max_tokens=384` for the MGD arms (whose generations are longer due to slower decoding under the mask). The MGD arms are capped at $n=15$ because rust-analyzer’s per-query latency degrades over a session

(problem 5 of an uncapped run took 953 s); we mitigate with a “restart provider every 5 problems” workaround and a 90 s wall-budget per problem, but cannot run the full $n=30$ within reasonable wall time on the current implementation.

[INCOMPLETE] $n=15$ vs $n=30$ for MGD arms; the budget cap doesn’t actually preempt synchronous `model.generate`, so budget-aborted problems still consume 3–7 minutes wall time each. We estimate §7 that fixing the rust-analyzer lifecycle issue would reduce MGD wall to under 2 hours for all 156 problems $\times$ all arms.

Result. Table 3.

Table 3: Five-arm pilot, Qwen2.5-Coder-1.5B-Instruct on MultiPL-E HumanEval Rust. Rollback arms run at $n=30$; MGD arms run at $n=15$ (see

[INCOMPLETE]

above). Right column: rollback arms restricted to the same 15 problems MGD ran on, for paired comparison.

Arm	n	pass@1	compile	mean wall	pass@1 (n=15 paired)
raw	30	0.533	0.700	1.5 s	0.667
rollback (mute)	30	0.667	0.800	2.1 s	0.733
rollback (instruct)	30	0.700	0.800	1.7 s	0.733
MGD (post-fix)	15	—	—	30.9 s	0.267
MGD + rollback (post-fix)	15	—	—	86.3 s	0.200

Headline. MGD reduces pass@1 from 0.667 (raw) to 0.267 on the matched 15-problem subset — a 40-percentage-point regression. MGD+rollback reduces it further to 0.200. Both rollback-only variants *improve* the baseline by 6.6 pp under the same conditions. This is the opposite of the direction the MGD paper reports for Java [1], by a wide margin.

4.3 Java reproduction check

To rule out a misreport of the published gains, we ran the MGD authors’ bundled evaluator (`evaluation_scripts/eval_results.py`) on their bundled inference outputs (`inference_results/dotprompts_results_sample.csv`). This does *not* re-run inference. It re-derives Table 1’s metrics from saved generations.

[INCOMPLETE] We have only verified the *evaluator + saved-outputs* half of the original experiment. We have **not** re-run the MGD inference pipeline ourselves on the DOTPROMPTS testcases. Doing so would require loading CodeGen-350M / SantaCoder and running their LogitsProcessor against multilspy + Eclipse JDT.LS for each testcase. Estimated effort: 1 day. See §8.

Result. On the bundled 27-method sample, 8 of 9 base/MGD configurations show CR improvement under MGD; 9 of 9 show NIM improvement. Magnitudes per Table 4.

Table 4: Java reproduction: paper’s published evaluator on the paper’s saved generations, sample subset only. “ Δ ” columns are absolute pp gains under MGD vs the matched base configuration.

config (base $\rightarrow$ MGD)	CR@6 base	CR@6 MGD	Δ CR	Δ NIM
CG-350M	66.7	66.7	+0.0	+22.2
CG-2B	44.4	66.7	+22.2	+33.3
CG-6B	44.4	77.8	+33.3	+22.2
SantaCoder	55.6	77.8	+22.2	+33.3
TD-3	22.2	44.4	+22.2	+11.1

The directions match Agrawal et al. [1] Table 1; magnitudes are larger on the sample due to small- n variance but qualitatively consistent. The MGD gains as reported by the bundled artifacts are real. The Rust regression we observe is therefore a property of our setup, not a misreport of theirs.

5 Diagnosis: why the Rust port fails

5.1 Bug: multilspy returns labels not identifiers

A controlled diagnostic at six representative trigger sites (e.g. `numbers.`, `result.`, `(numbers[i] - numbers[j]).`) revealed that the “legal completions” returned by our provider were LSP *labels* rather than bare identifiers:

```
'to_owned() (as ToOwned)'      # has trailing trait disambiguation
'iter()'                       # has function-call notation
'drain(...)'                  # has Unicode ellipsis denoting args
'dbgr'                         # postfix snippet, not a real method
'refm'                         # postfix snippet
'len()'
```

The cause: `multilspy`’s `language_server.py` sets `completion_item["completionText"] = item["label"]` when the LSP response contains a `label` field, falling back to `insertText` only if absent. For `rust-analyzer` responses, `label` is always present and is always the decorated display string. The reference `DereferencesMonitor` expects the bare identifier; our port (faithfully consuming `completion["completionText"]`) was matching against decorated strings.

5.2 Fix

We strip non-identifier characters from completion text and filter by `CompletionItemKind` to drop snippets, keywords, and other non-identifier kinds:

```
def _extract_leading_identifier(text):
    n = 0
    for c in text:
        if c.isalnum() or c == '_': n += 1
        else: break
    return text[:n]

IDENTIFIER_KINDS = {2, 3, 4, 5, 6, 7, 8, 9, 10, 13, 20, 21, 22, 25}
# Method, Function, Constructor, Field, Variable, Class, ...
```

After the patch, the diagnostic returns clean Rust identifiers (`as_mut_ptr`, `iter_mut`, `as_ptr`, `len`, `iter`, ...). All nine smoke tests still pass.

5.3 Re-run after fix

Table 5: MGD pilot before and after fix; same 15-problem subset on Qwen2.5-Coder-1.5B.

Arm	pass@1 (pre-fix)	pass@1 (post-fix)
raw (baseline)	10/15 (0.667)	10/15 (0.667)
mgd	3/15 (0.200)	4/15 (0.267)
mgd + rollback	5/15 (0.333)	3/15 (0.200)

The fix improves the `mgd` arm by one problem and *regresses* `mgd+rollback` by two. Inspecting the regressions: the pre-fix code was working partly by accident. Decorated suffixes (`iter()`, `to_owned()` (as `ToOwned`), etc.) contained additional characters beyond the identifier, so the mask’s prefix-match surface was *larger* than it should have been. With identifier-only suffixes, the mask is tighter, more decisions are forced, and the small model’s failure surface widens. The fix is correct (the legal set is now what the algorithm intends) but it sharpens rather than reverses the harm.

6 Discussion

6.1 Why rollback’s effect depends on baseline strength

We observe three failure modes for rollback events:

1. **Discarded code includes good prefix.** Rollback truncates to the last OK checkpoint, which may be many statements before the actual error. The LM must regenerate not only the bad statement but everything since the last clean state.
2. **Oscillation.** Especially in weak models, the re-prompt produces the same kind of mistake, the rollback fires again, and eventually the 8-rollback cap is hit with an empty buffer. Among rollback-affected nemotron problems, 7 of 11 hit the cap.
3. **Intercepting self-correcting paths.** The LM emits code that mid-generation looks broken (e.g., `let mut result = Vec::new();` with unknown element type) but would have resolved on the natural path once the variable is used. Rollback prevents the resolution.

For strong baselines, $\sim 95\%$ of problems already succeed on the raw arm. The rollback arm fires its mask anyway on $\sim 10\%$ of problems (false positives from the `unreachable!()` closer interaction with type inference), so a few clean wins are flipped to failures. This is exactly the Qwen-9B regression.

6.2 Why MGD on Rust regresses where MGD on Java succeeds

Four orthogonal differences between this pilot and the MGD paper’s Java evaluation:

Table 6: Axes of difference between our setup and the MGD paper.

Axis	MGD paper	this pilot
LSP	Eclipse JDT.LS (permissive on partial code)	rust-analyzer (strict, infer-dependent)
Task	dereference completion (1 trigger / 30 tokens)	whole-function (5–30 triggers / 200+ toks)
Metric	CR + NIM (identifier-match)	pass@1 + compile (functional + build)
Sampling	score@6 (best-of-6 nucleus)	n=1 sample

Three of the four (task, metric, sampling) systematically advantage the Java setup — fewer mask events per testcase, an identifier-level metric instead of a functional one, and a best-of-6 aggregation that absorbs single bad-mask events. The LSP axis we cannot decompose further with current data, but we expect it to dominate: rust-analyzer’s type-inference is strict, so partial code routinely lacks the type information the LSP needs to return a useful completion set. With the `unreachable!()` closer in particular, variables whose types are pinned only by later use return empty or wrong-typed completion sets, and the mask forces incorrect choices.

6.3 Implication for “LSP-guided decoding” as a method

The two methods occupy opposite points on a precision-recall axis. **MGD** is high-precision-low-recall on what it constrains: it demands the next identifier be valid at the type level, but only at . sites. **Rollback** is low-precision-high-recall: it catches anything that breaks `cargo check` at a boundary, but fires sparsely. Neither dominates the other; they target different failure modes. Our hypothesis going into the pilot was that combining them (arm iv) would be additive — catching dereference errors early with MGD’s mask and structural errors late with rollback. Our evidence does not support that hypothesis: MGD+rollback underperforms both raw and rollback-only on Qwen2.5-Coder-1.5B. We attribute this to MGD’s mask creating most of the failures that rollback then can’t unstick.

7 Limitations

Single MGD model. Our MGD arms were measured on one model (Qwen2.5-Coder-1.5B-Instruct) at $n=15$. The MGD paper used five models from 350M to 175B; our finding may be specific to small models. A larger model with broader probability mass over tokens may absorb the mask gracefully where a 1.5B model does not.

[INCOMPLETE] Add at least one 7B-class HF model to the MGD comparison. Estimated effort: 1 day if the rust-analyzer lifecycle issue is fixed first.

Single language. All pilots are on Rust. The MGD paper’s Java result may or may not transfer to Java when re-run end-to-end on the same models we used; we have only verified the eval half.

n=15 cap on MGD arms. The MGD arms ran at $n=15$ rather than $n=30$ because of rust-analyzer query-latency degradation across a session under multilspy. The non-MGD arms ran at $n=30$. Paired comparisons subsample the rollback arms to the same 15 problems for fairness, but a power calculation suggests $n=30$ would already be borderline for the headline claim; a full $n=156$ run is needed.

[INCOMPLETE] Replace multilspy’s `open_file` lifecycle with the `lsp-client` package to fix the latency growth, then re-run all arms at $n=156$. Estimated effort: 1 engineering day plus ~ 2 hours of compute.

vLLM not used. We initially targeted vLLM for higher throughput; v1’s engine-subprocess broke our `LogitsProcessor` closure (§3.5). HF transformers works but is slower. A production-quality MGD evaluation would use vLLM’s class-based `LogitsProcessor` interface with shared-memory state, or vLLM v0 in single-process mode.

[INCOMPLETE] Either path is doable but delicate; we left this for a follow-up.

Budget cap doesn’t preempt sync work. Our 90 s per-problem wall budget uses `asyncio.wait_for`, which cannot interrupt `model.generate`. So a budget-exceeded MGD problem still runs 3–7 minutes wall time before yielding. A hard kill via signal would preempt but would also leak GPU memory.

[INCOMPLETE] Implement thread-based hard timeouts with proper cleanup.

Java reproduction is eval-only. We re-ran the published evaluator on the published outputs (§4.3). We did **not** re-run the MGD inference pipeline end-to-end on the DOTPROMPTS testcases.

[INCOMPLETE] Run CodeGen-350M-multi with the reference `DereferencesMonitor` through multilspy + Eclipse JDT.LS on the first 100 DOTPROMPTS testcases; check that regenerated outputs reproduce the published CR/NIM numbers.

No DOTPROMPTS-equivalent for Rust. We did not build a Rust analog of DOTPROMPTS (real-repo method truncated at a `.`). This is the only experiment that would isolate the language/LSP axis from the task-framing axis in the MGD comparison.

[INCOMPLETE] Curate $\sim 1,000$ Rust dereference prompts from real-world crates; evaluate the same models with and without MGD on NIM + CR (drop-back-into-source build). Estimated effort: 1 week.

No closer ablation. We use `unreachable!()` as the universal-return shim for partial-buffer type-checking. This hides return-type errors at intermediate boundaries. An alternative (closure-wrapping) would not have this hiding effect but would require careful `return`-statement handling.

[INCOMPLETE] Pilot a closure-wrap closer and compare its rollback-rate / compile-rate against the `unreachable!()` closer.

No score@k for our arms. The MGD paper reports `score@1..6`; we report only $n=1$.

[INCOMPLETE] Re-run with 6 samples per problem under nucleus sampling and report `pass@k` and `compile@k` for $k \in [1, 6]$.

8 Conclusion and next steps

We pilot-compared four arms (raw, MGD, rollback, MGD+rollback) of code generation on Rust + rust-analyzer. Three pilot-scale findings:

1. **Rollback verification is a baseline-quality-dependent tradeoff** (+6.7 pp compile rate on weak/mid models, −10 pp pass@1 on a strong baseline).
2. **MGD’s Java success does not transfer to Rust on a 1.5B model under our pilot conditions** (−40 pp pass@1 vs raw). The result is robust to fixing one real port-side bug.
3. **The four orthogonal axes of difference** (language, LSP, task framing, sampling regime) between our pilot and the MGD paper each plausibly contribute. The single experiment that would isolate them — a Rust DOTPROMPTS analog at score@6 — has not been built.

This is a pilot. The most informative single follow-up experiment is building a Rust DOTPROMPTS analog and running MGD on it: if MGD still hurts under matched methodology, the LSP axis dominates; if MGD helps, the gap is task-framing. The other immediate next steps are listed inline as

[INCOMPLETE]

-tagged blocks throughout the paper.

Acknowledgments

This work was done as a CMSC 25750 quarter project at the University of Chicago. Compute donated by [redacted for review].

References

- [1] Lakshya A Agrawal, Aditya Kanade, Navin Goyal, Shuvendu Lahiri, and Sriram Rajamani. Monitor-guided decoding of code LMs with static analysis of repository context. In *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2306.10763>. arXiv:2306.10763.
- [2] Lakshya A Agrawal et al. multilspy: Cross-platform python LSP client library, 2023. Part of monitors4codegen; <https://github.com/microsoft/monitors4codegen>.
- [3] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. Prompting is programming: A query language for large language models, 2023. arXiv:2212.06094.
- [4] Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q Feldman, et al. MultiPLE: A scalable and polyglot approach to benchmarking neural code generation. *IEEE Transactions on Software Engineering*, 2023. arXiv:2208.08227.
- [5] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Sean Welleck, Bodhisattwa Prasad Majumder, Shashank Gupta, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, 2023. arXiv:2303.17651.
- [6] Theo X Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. Is self-repair a silver bullet for code generation? In *International Conference on Learning Representations*, 2024. arXiv:2306.09896.
- [7] Gabriel Poesia, Alex Polozov, Vu Le, Ashish Tiwari, Gustavo Soares, Christopher Meek, and Sumit Gulwani. Synchromesh: Reliable code generation from pre-trained language models. In *International Conference on Learning Representations*, 2022. arXiv:2201.11227.
- [8] Tal Ridnik, Dedy Kredo, and Itamar Friedman. Code generation with AlphaCodium: From prompt engineering to flow engineering, 2024. arXiv:2401.08500.
- [9] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. PICARD: Parsing incrementally for constrained auto-regressive decoding from language models. In *Empirical Methods in Natural Language Processing*, 2021. arXiv:2109.05093.

- [10] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023. arXiv:2303.11366.
- [11] Shubham Ugare, Tarun Suresh, Hangoo Kang, Sasa Misailovic, and Gagandeep Singh. Syn-Code: LLM generation with grammar augmentation. In *International Conference on Machine Learning*, 2024. arXiv:2403.01632.
- [12] Brandon Willard and Rémi Louf. Outlines: Guided generation, 2023. arXiv:2307.09702.

A vLLM v1 LogitsProcessor IPC issue

vLLM 0.20.1’s v1 architecture runs the engine in a separate process, and registers logits-processor classes via the `LLM(logits_processors=[...])` parameter. When the processor’s `new_req_logits_processor` is called per request, the returned closure is dispatched to the engine subprocess for invocation each generation step. The closure can capture state, but mutations to that state in the subprocess do not propagate back to the main process. Our diagnostic with `os.getpid()` confirmed this: trigger counts incremented in the engine subprocess; the `_GLOBAL_TELEMETRY[req_id]` dict in the main process remained at zero.

Additionally, the `RustAnalyzerCompletionsProvider` owns an `asyncio` loop and a `threading.Lock`; it cannot be safely pickled to the subprocess. The closure ends up referencing a torn-down provider in the subprocess and returns empty completion lists silently.

We worked around both by switching to HF transformers, which runs `model.generate` in-process.

B Smoke test list

Nine unit tests; all pass post-fix.

- Boundary detector: simple semicolons, string with semicolon, semicolon inside parens (suppressed), raw string with semicolon.
- Cargo check: clean code, type mismatch caught.
- Code state machine: clean check creates checkpoint, error blocks, rollback restores buffer.